

Road Network Description File Labelling

Road Network Description File Labelling

1. Course Content
2. Preparation Work
 - 2.1 Start Agent
 - 2.2 Road Network Labelling
 - 2.3 Start ros2_kilted docker container (Skip for PI5 & RDK mainboards)
 - 2.4 Start Road Network Labelling Node
 - 2.5 Add Waypoints
 - 2.6 Add Edges
3. Save Road Network Description File
4. Load and Modify Road Network Description
 - 4.1 Load Road Network Description File
 - 4.2 Edit Road Network

1. Course Content

1. Use route_tool tool to label `.geojson` format files that describe road networks

2. Preparation Work

2.1 Start Agent

- Use the following command to start microROS agent to connect to the chassis

[!WARNING]

If microROS agent is already started, close the previous agent first. Note that this command is different from the previous microROS agent startup command

```
start_kilted_agent
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1763535321.445792] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1763535321.449556] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1763535321.452876] info | ProxyClient.cpp | create_subscriber | s
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1763535321.457712] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1763535321.461959] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1763535321.466141] info | ProxyClient.cpp | create_subscriber | s
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1763535321.473112] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

- Modify ROS communication middleware DDS

[!WARNING]

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Open ~/.bashrc, uncomment this line, and change it to the DDS communication middleware as shown in the figure (only the road network planning and navigation chapter needs to use cyclondds, **please restore the environment variables after completing the road network chapter!!**)

```
vim ~/.bashrc
```

- RDK & PI5 mainboards

```

131 #ROSMaster-M3 Code space environment variables
132 export PYTHONPATH="/home/sunrise/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
133 export ROS_DOMAIN_ID=16
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 #unset RMW_IMPLEMENTATION
136 export ROBOT_TYPE="ROSMaster-M3"
137 export MULTI_ROBOT="" #robot1 or robot2
138 #=====雷达和相机型号=====
139 #=====Radar and camera models=====
140 #=====
141 #单雷达: single          双雷达: dual
142 #Single radar: single    Dual radar: dual
143 export LASER_TYPE=single
144 #export LASER_TYPE=dual
145

```

- ORIN mainboard

```

133 export ROS_DOMAIN_ID=30
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 export ROBOT_TYPE="ROSMaster-M3"
136 export MULTI_ROBOT=""
137

```

```

:wq #Save and exit
source ~/.bashrc #Update environment

```

2.2 Road Network Labelling

- When marking the road network, we need to know the specific position of the vehicle in the map, so we need to start the navigation node on the host first. Through the position of the base_footprint coordinate system, we can know the specific position of the vehicle in the map
- (For PI5 users, start in the `m3` container)

```

ros2 launch m3_bringup nav2.launch.py map:=map.yaml pose_map:=map
localization_mode:=slam_toolbox gui:=False

```

[!NOTE]

Startup parameter description

- **map** The grid map name to be loaded, here we choose the map saved in the previous lesson
- **pose_map** The pose map file name to be loaded **without file suffix**, used for more accurate and fast positioning of the vehicle
- **localization_mode** Global localization method, here we choose `slam_toolbox` when marking the road network, which can achieve higher precision global positioning using the pose map
- **gui** Whether to start rviz, here we choose not to start, it will be started later in the docker container

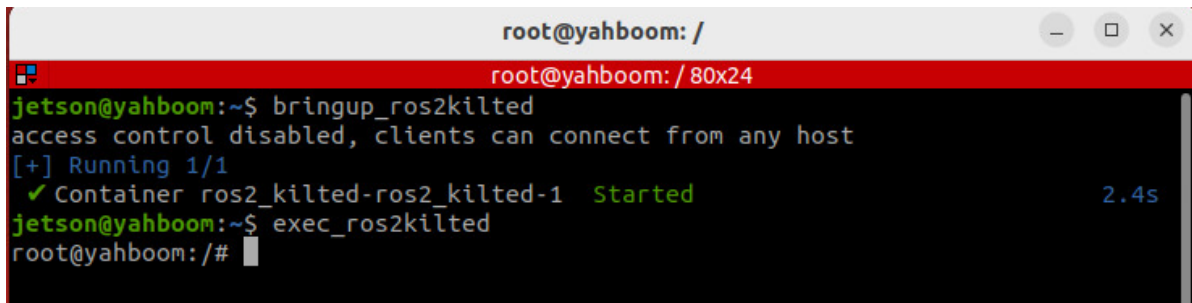
2.3 Start ros2_kilted docker container (Skip for PI5 & RDK mainboards)

- Start ros2_kilted container (only need to start once, no need to repeat)

```
bringup_ros2kilted
```

- Enter ros2_kilted container terminal

```
exec_ros2kilted
```



```
root@yahboom: /  
root@yahboom: / 80x24  
jetson@yahboom:~$ bringup_ros2kilted  
access control disabled, clients can connect from any host  
[+] Running 1/1  
✔ Container ros2_kilted-ros2_kilted-1 Started 2.4s  
jetson@yahboom:~$ exec_ros2kilted  
root@yahboom:/#
```

2.4 Start Road Network Labelling Node

- Start in ros2_kilted container terminal

(ORIN mainboard)

```
ros2 launch road_net_route route_tool.launch.py yaml_filename:=map.yaml
```

(PI5 & RDK mainboards) Only need to start rviz, visualization is started on the accompanying virtual machine Rosmaster-RoadNet-Ubuntu24.04,

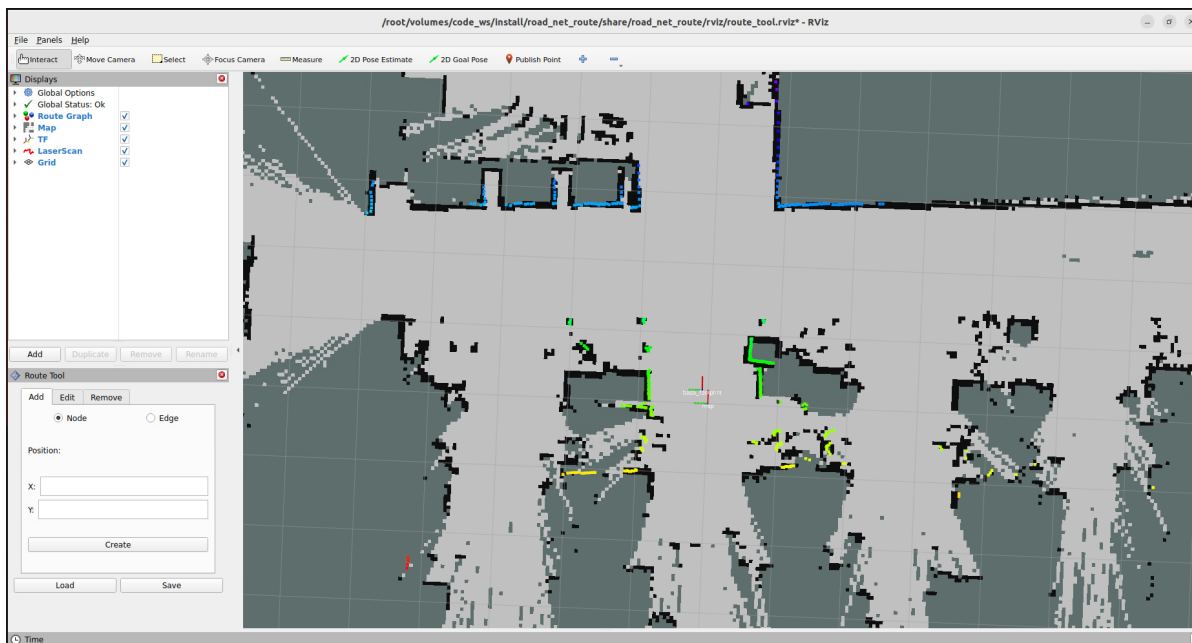
```
#Virtual machine side  
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/route_tool.rviz
```

- **yaml_filename** parameter is the grid map file path of the road network to be labeled, here we use the map built in the previous lesson

[!NOTE]

The host's map will be saved in the `~/map` path by default. This path will be automatically mounted to the `/root/map` directory in the ros2_kilted container, so the map files saved on the host can be used directly in the ros2_kilted container

- The map and two coordinate systems will be displayed in rviz, where map is the map coordinate system, and base_footprint is the coordinate system of the projection point of the vehicle chassis center on the ground. Through base_footprint, we can know the accurate position of the vehicle in the map in real time

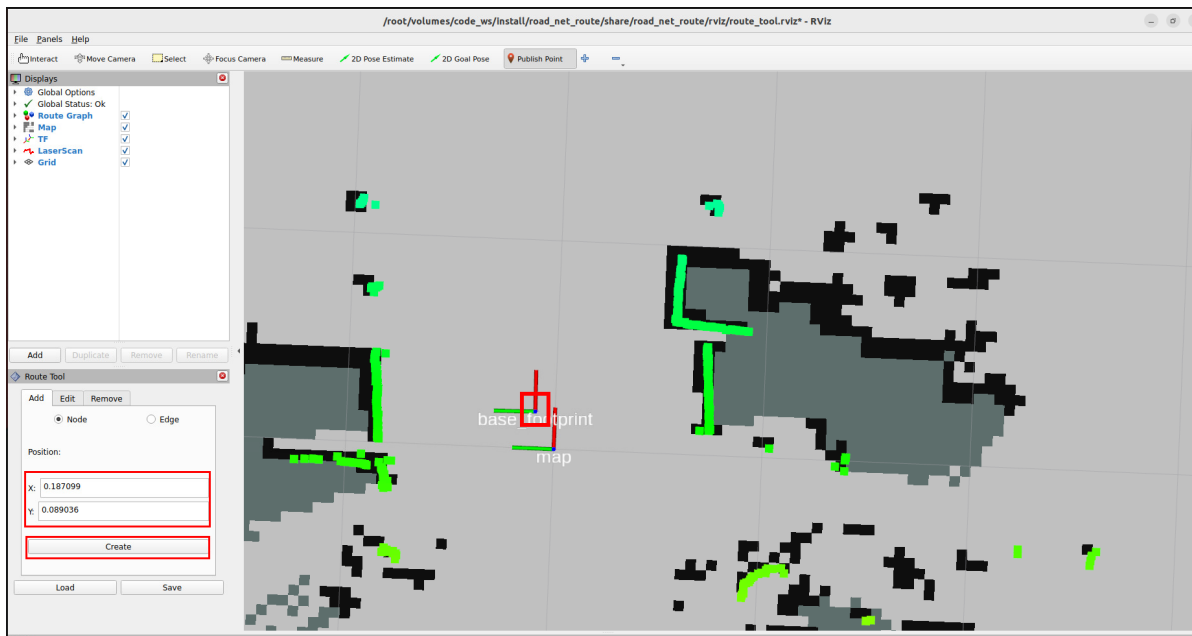


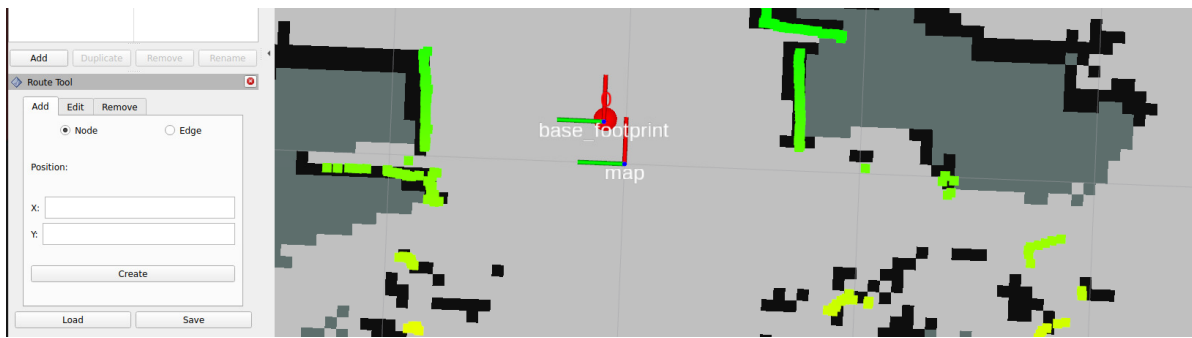
2.5 Add Waypoints

- Click the `Publish Point` tool in the rviz toolbar to enter waypoint selection state



- Then click the left mouse button at the center blue dot of the `base_footprint` coordinate system. The `position` in the `Route Tool` tab in the lower left corner will automatically fill in the current point's coordinates, then click `Create` to create a waypoint



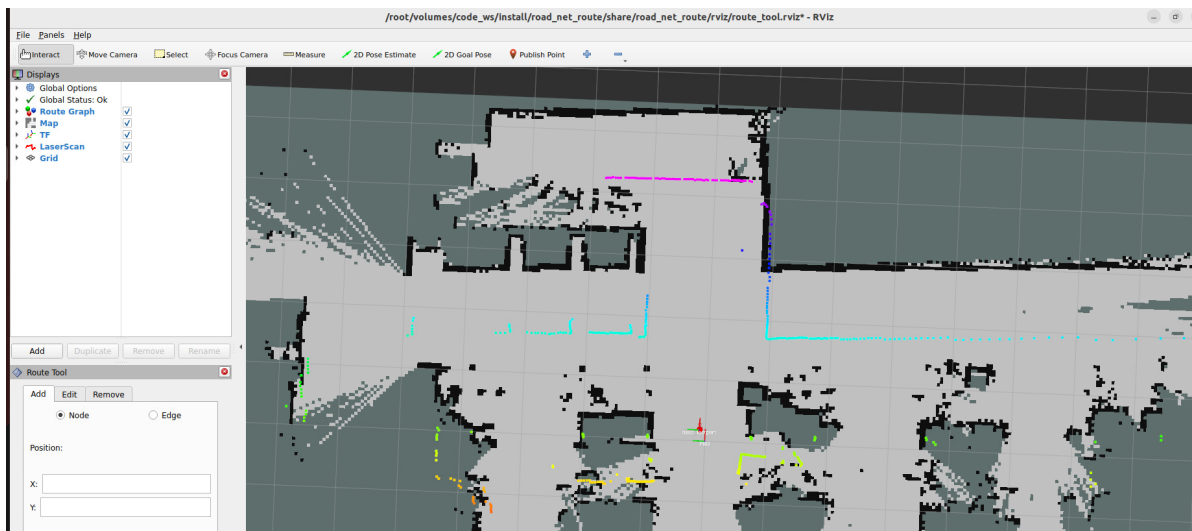


- Then we move the vehicle to the next position where we need to mark a waypoint. In this demonstration, we directly move the vehicle to the next position where we need to mark a waypoint

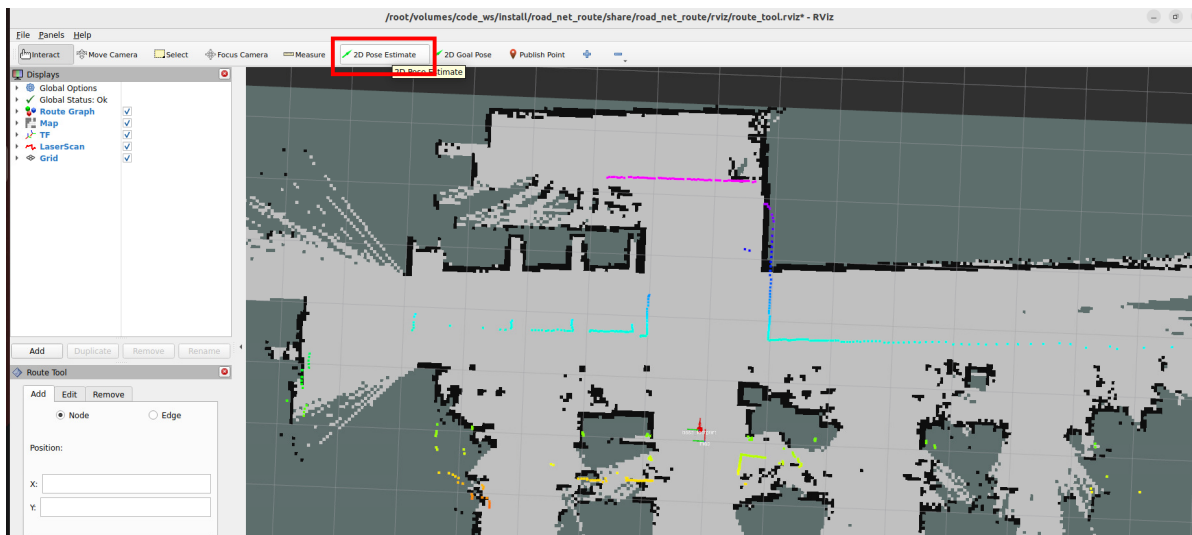
[!TIP]

Here you can move the robot vehicle through keyboard control, gamepad control, `2D Goal Pose` navigation in rviz, or even directly move the vehicle to the next position to be marked, because with the pose map, you can quickly and accurately locate the global position of the vehicle

- As you can see, because the vehicle was moved directly, the global positioning is lost, and the laser contour no longer matches the map environment features. Here we can use the pose map for fast relocalization



- Click the `2D Pose Estimate` tool in the rviz toolbar



- Then give an estimated position near the approximate position of the robot vehicle



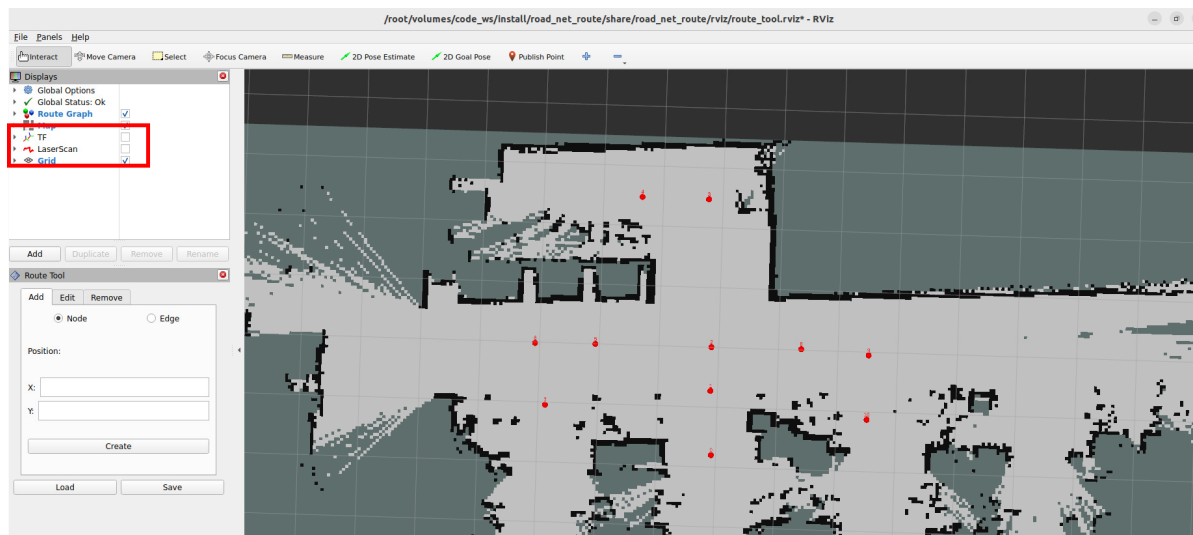
- After that, the accurate positioning of the vehicle in the global map will be immediately restored

[!TIP]

- The principle of fast relocalization here is to use slam_toolbox pure localization and pose map for fast scan-map matching
- **Note:** When you observe that the **laser contour and map environment features have deviations** during the subsequent waypoint marking process (due to accumulated odometry errors, wheel slippage, manual movement, and other factors causing local positioning errors), you can use this method to quickly restore the accurate positioning of the vehicle.

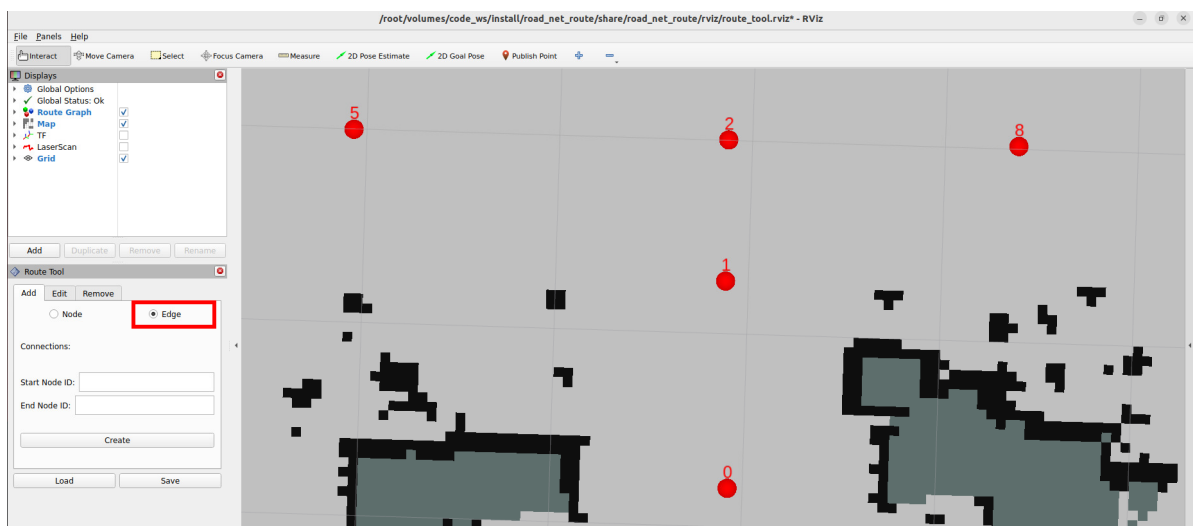


- Repeat the above steps until all required waypoints are completed (mark waypoints according to your actual needs)
- After marking is complete, you can close TF and LaserScan on the left side to check the waypoints



2.6 Add Edges

- Select Edge in the Route Tool tab



- Fill in the starting waypoint in **Start Node ID** and the ending waypoint in **End Node ID**, then click Create to create an edge between waypoints

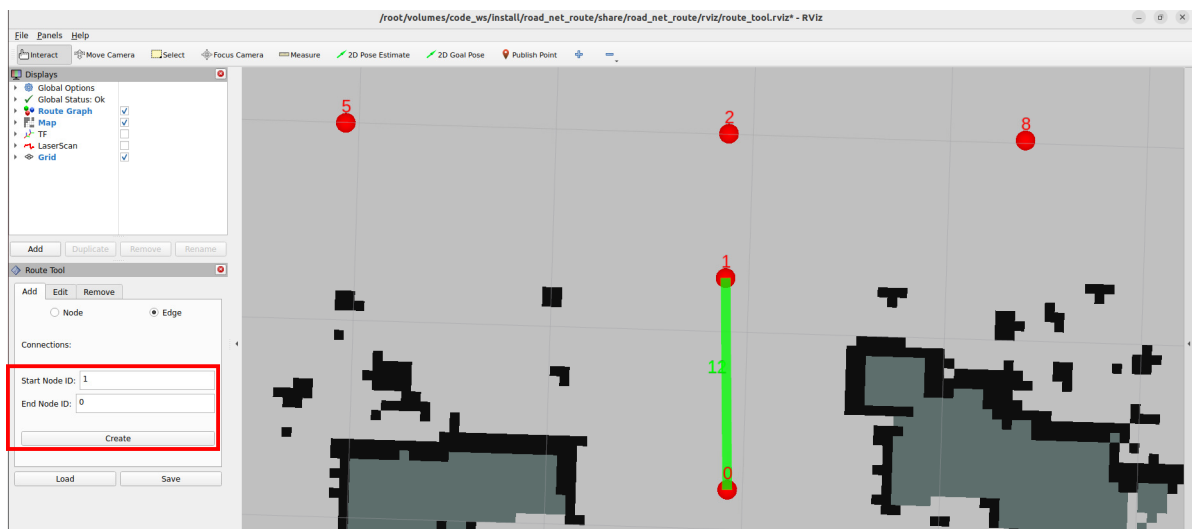
[!IMPORTANT]

Attention!!!

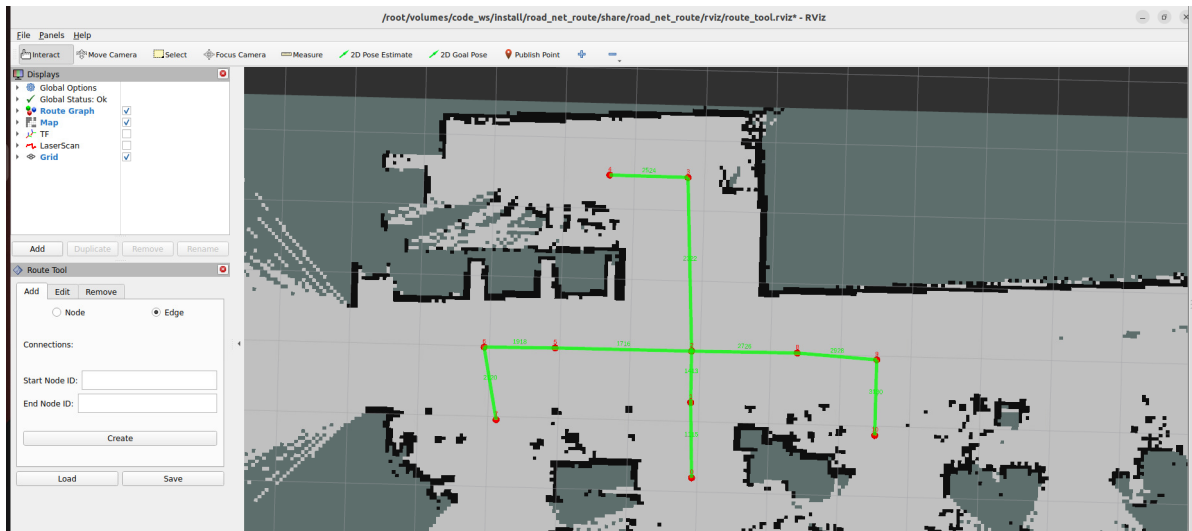
A road network is a topological structure composed of points and edges, and edges are directional. For example, creating an edge 0---->1 here is unidirectional. If you need a bidirectional edge, you also need to create an edge 1---->0.



- Then we create the edge 1-0, so that vehicles can travel in both directions between waypoints 0 and 1

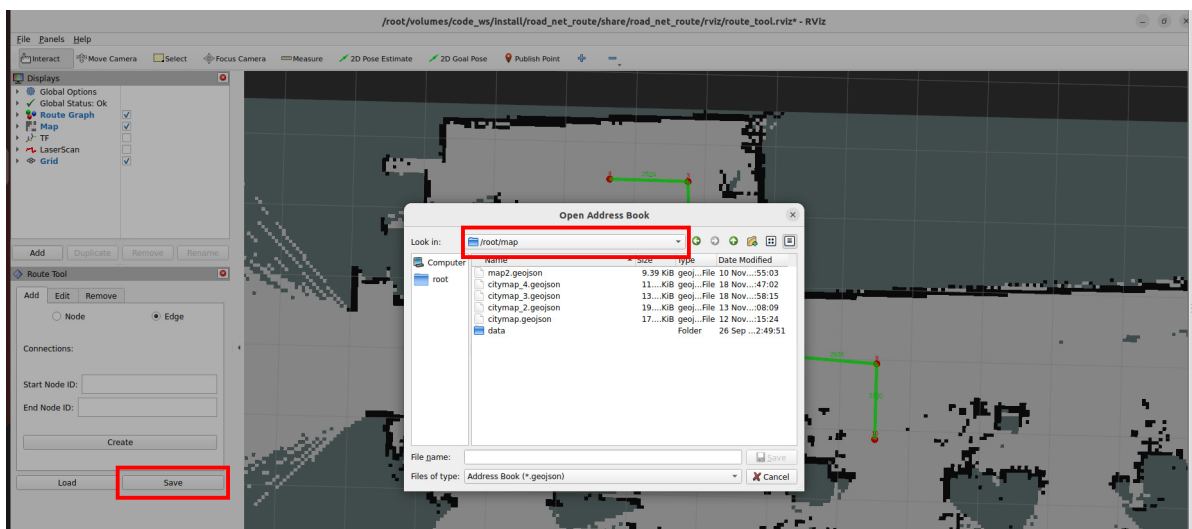


- Repeat the above steps to complete the creation of all edges (whether the edges created here are unidirectional or bidirectional depends on actual scenario requirements)

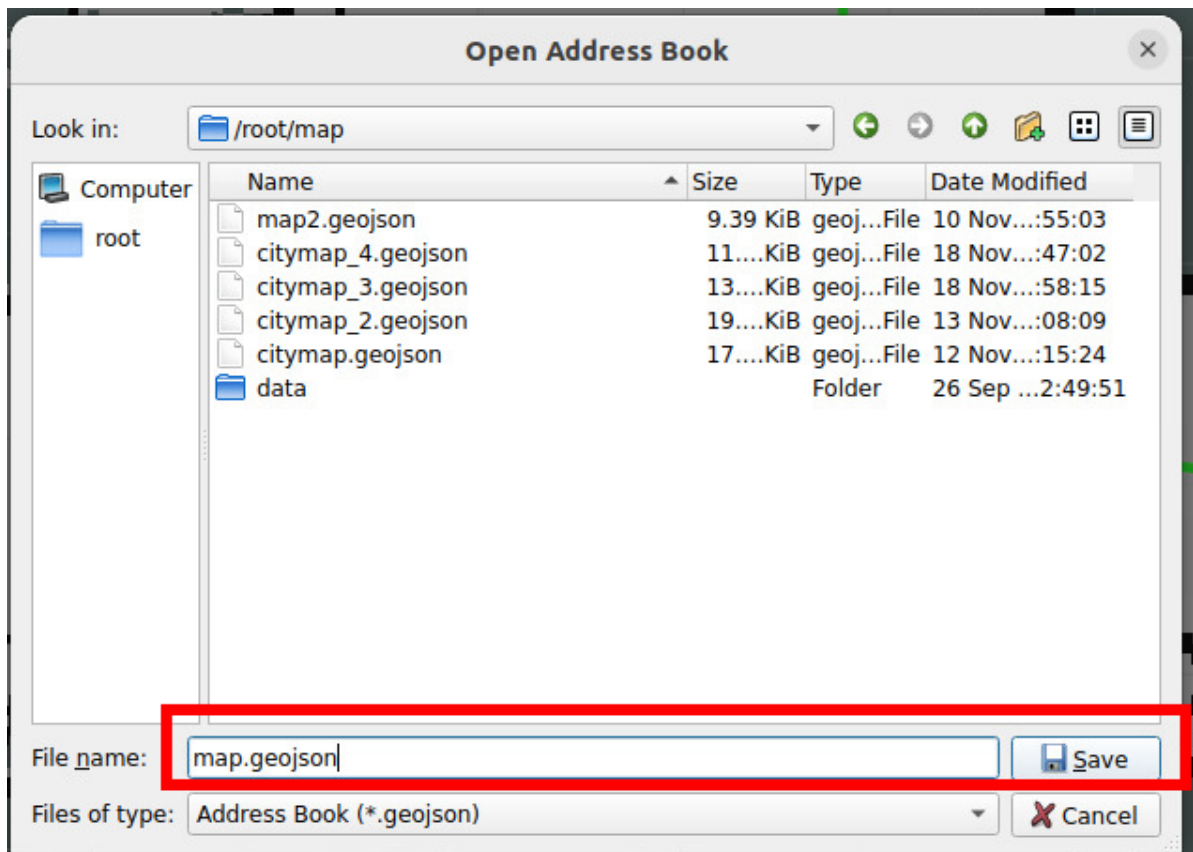


3. Save Road Network Description File

- Click save in the `Route Tool` tab, then select the path to save the file in the pop-up window. Here we choose the `/root/map` path, because this path is bound to the host's `~/map` path, and you can directly view the saved files on the host later



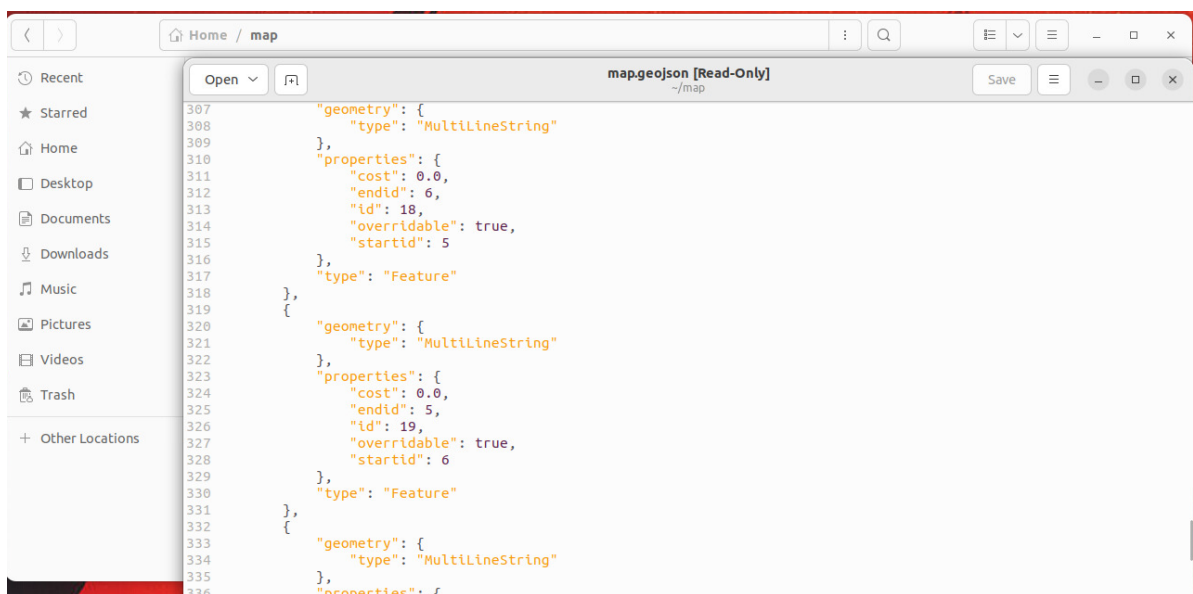
- Then fill in the file name in File name. It is recommended to keep it consistent with the grid map name for easy distinction. The file suffix must be `.geojson`. Finally, click save to save the road network description file



- After successful saving, the terminal will print prompt information

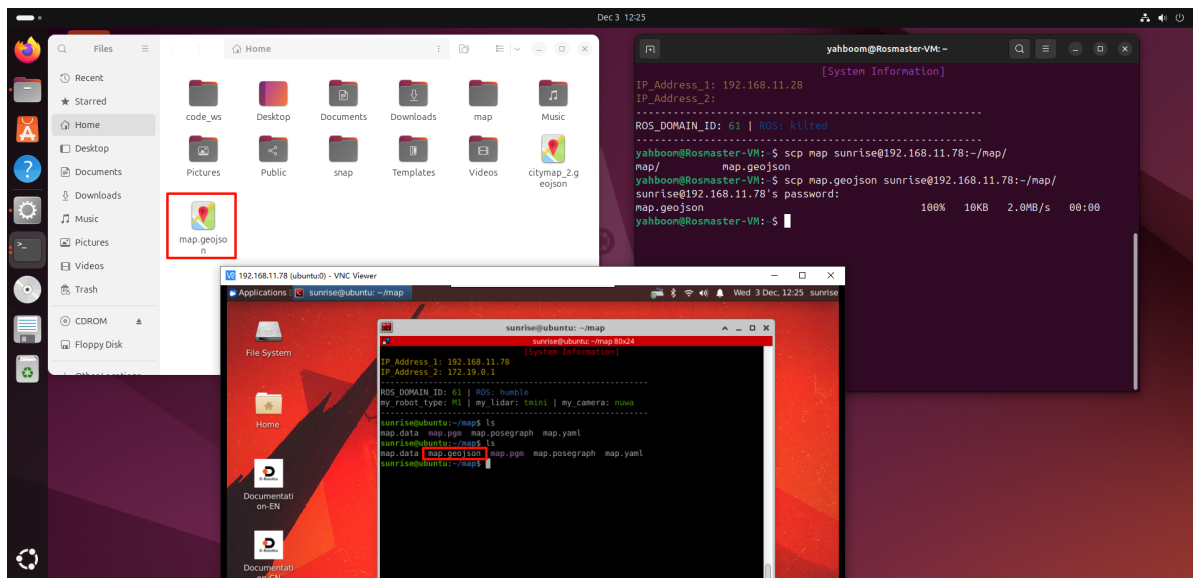
```
[rv122-1] [INFO] [1763543213.88111168] [route_tool_node]: Save graph to:
[rv122-1] [WARN] [1763543213.881559940] [route_tool_node]: The graph filepath was not provided and no default was specified. Failed to save the route graph.
[rv122-1] [INFO] [1763543726.803456606] [route_tool_node]: Save graph to: /root/map/map.geojson
[rv122-1] [INFO] [1763543726.803604931] [route_tool_node]: Saving graph file from /root/map/map.geojson, by parser nav2_route::GeoJsonGraphFileSaver
```

- Then we can directly view the map.geojson road network description file from the host's `~/map` path. The file contains JSON format descriptions of waypoint and edge information



- (PI5 & RDK mainboards) Transfer road network file

Use the `scp map.geojson sunrise@192.168.11.78:~/map/` command (fill in actual username and ip) to directly transfer to the host's `~/map` path, because this path is bound to the `/root/map` path in the docker container. You can also drag it out from the virtual machine and then transfer it to the host.



4. Load and Modify Road Network Description

4.1 Load Road Network Description File

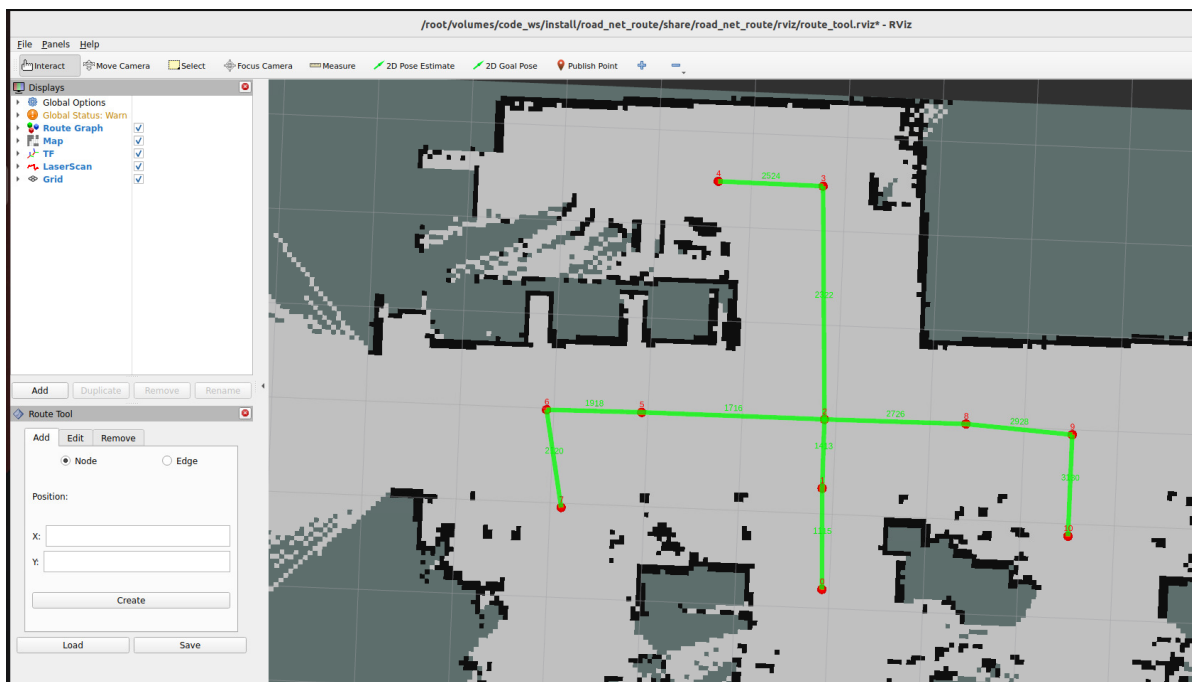
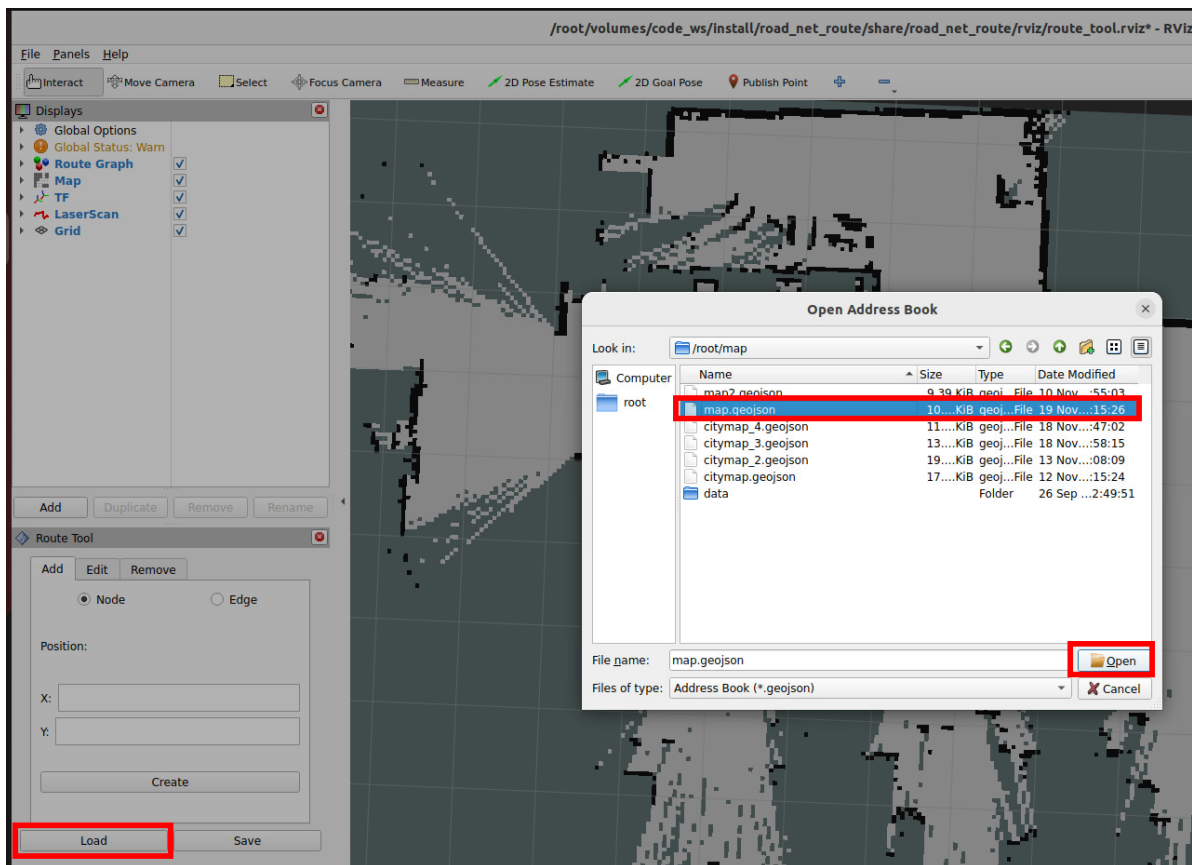
- If you need to modify the previous road network description file later, such as adding, removing, editing points, or edges, you can load the previous road network description file for editing
- Start in `ros2_kilted` container terminal (Skip for PI5 & RDK mainboards)

```
ros2 launch road_net_route route_tool.launch.py yaml_filename:=map.yaml
```

- **yaml_filename** parameter is the grid map file path of the road network to be labeled
- (RDK mainboard) Only need to start rviz, visualization is started on the accompanying virtual machine `Rosmaster-RoadNet-Ubuntu24.04`,

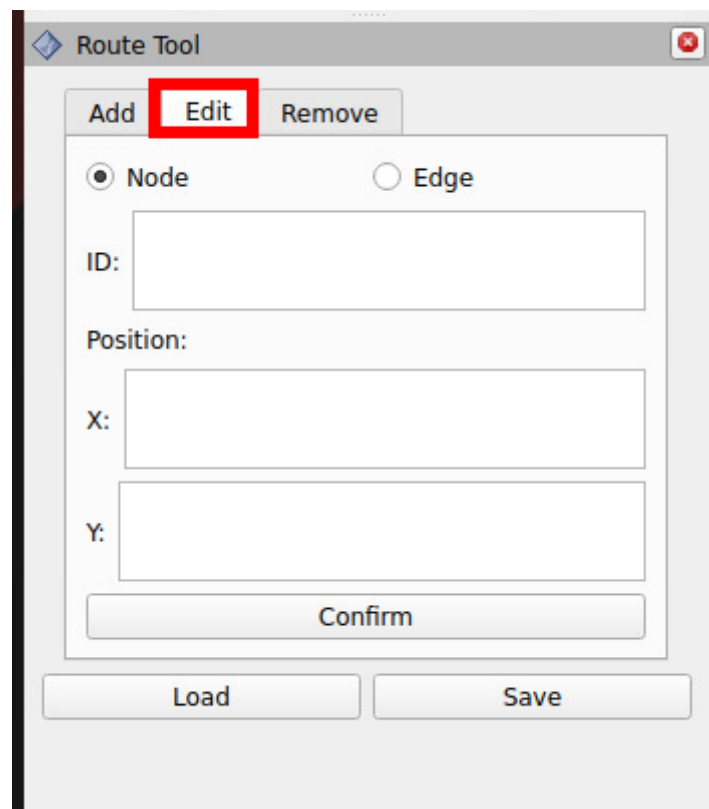
```
#Virtual machine side
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/route_tool.rviz
```

- ☒ Click Load in the `Route Tool` tab, then select the road network description file to be edited, and click `open` to load



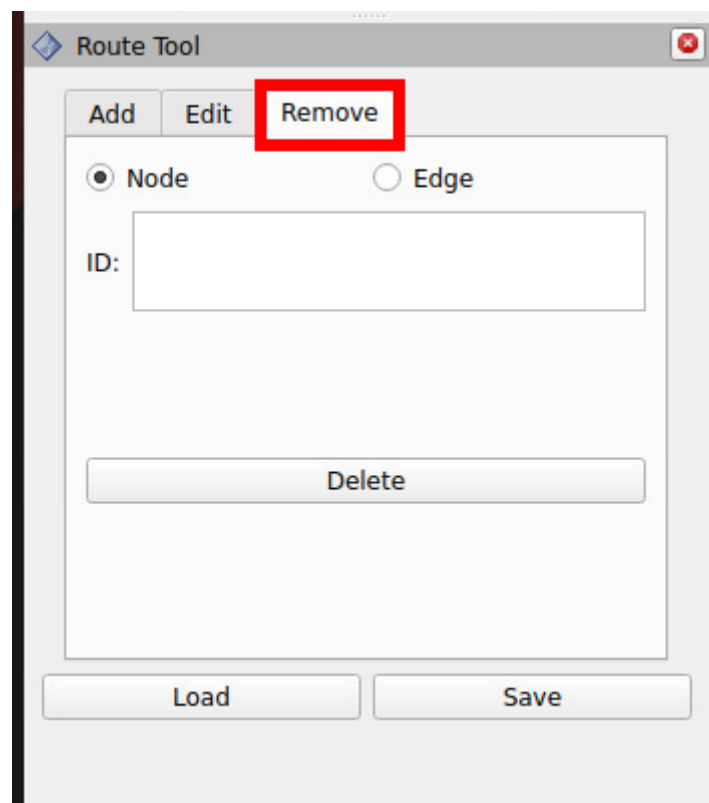
4.2 Edit Road Network

- If you need to add new waypoints and edges, the operation is the same as the previous steps
- Click **Edit** to re-edit information about existing waypoints or edges



The image shows a software window titled "Route Tool". At the top, there are three buttons: "Add", "Edit", and "Remove". The "Edit" button is highlighted with a red rectangular box. Below these buttons, there are two radio buttons: "Node" (which is selected) and "Edge". Under the "Node" radio button, there is a text input field labeled "ID:". Below the "ID:" field, there is a section labeled "Position:" which contains two text input fields, one labeled "X:" and one labeled "Y:". At the bottom of this section is a button labeled "Confirm". Below the "Confirm" button, there are two more buttons: "Load" and "Save".

- Click **Remove** to remove existing waypoints or edges



The image shows the same "Route Tool" software window. In this view, the "Remove" button at the top is highlighted with a red rectangular box. The "Node" radio button remains selected. The "ID:" text input field is present. Below the "ID:" field, there is a button labeled "Delete". The "Load" and "Save" buttons at the bottom are also visible.